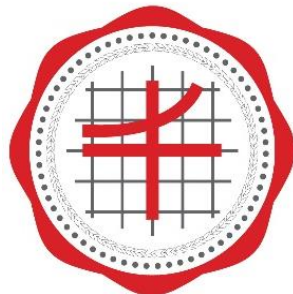


CPE 121 Mobile Application Developments

บทที่ 7

การจัดการข้อผิดพลาด (Exception Handling)



Faculty of Engineering

SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.

Department of Computer Engineering

การจัดการความผิดพลาด

การเขียนโปรแกรมจะมีข้อผิดพลาดที่ไม่ได้คำนึงถึงตั้งแต่แรกในการเขียนโปรแกรม ซึ่งอาจทำให้การทำงานของโปรแกรมล้มเหลว เช่น

- การหารด้วยศูนย์
- การเข้าถึงข้อมูลอาร์เรย์ในตำแหน่งที่ไม่มีอยู่จริง
- การป้อนข้อมูลผิดประเภท เป็นต้น

โดยทั่วไปการแก้ไขปัญหาสามารถทำได้หลายวิธี เช่น การตรวจสอบข้อผิดพลาดด้วยคำสั่ง **If – else**

ภาษา Java จะมีกลไกในการดักจับด้วยการโยน (Throw) เมื่อตรวจพบความผิดพลาดที่เกิดขึ้นในขณะที่โปรแกรมทำงานให้กับส่วนของชุดคำสั่งที่ทำหน้าที่ดังกล่าว โดยใช้งานคลาส Throwable ซึ่งมีคลาสลูก 2 คลาสด้วยกัน คือ คลาส **Error** และ คลาส **Exception**



Error และ Exception

Error คือ ความผิดพลาดที่อาจจะเกิดจาก Syntax Error สามารถตรวจพบได้ง่ายตั้งแต่ตอนคอมไพล์โปรแกรม หรือ เกิดจาก Logical Error ที่ใช้ตรรกะในชุดคำสั่งไม่ถูกต้อง ซึ่งตรวจพบได้ยาก

Exception คือ เหตุการณ์ผิดปกติที่เกิดขึ้น ส่งผลให้การทำงานของโปรแกรมไม่เป็นไปตามขั้นตอนที่กำหนดไว้ เช่น

- การหารด้วยศูนย์
- การเปิดไฟล์ที่ไม่มีอยู่จริง
- การกรอกข้อมูลที่ผิดพลาด



ประเภทของ Error

Error	คำอธิบาย
AWTError(java.awt)	Error ทั่วไปที่พบ ซึ่งแสดงว่า เกิดปัญหาขึ้นกับ Package ชื่อ "java.awt"
ClassFormatError	Format ของ Bytecode ใน Class File มีปัญหา
Error(java.lang)	Root ของ Error Hierarchy
ExceptionInInitializerError(java.lang)	เกิด Exception ขึ้นใน Static Initializer
IllegalAccessError(java.lang)	มีการเรียกใช้ Class, Method หรือ Variable ที่ไม่สามารถใช้ได้
IncompatibleClassChangeError(java.lang)	มี Operation ที่ไม่ถูกต้องเกิดขึ้นกับ Class
InstantiationError(java.lang)	มีการ Initialize Abstract Class หรือ Interface
InternalError(java.lang)	เกิดปัญหาขึ้นกับ Interpreter
NoClassDefFoundError(java.lang)	ไม่พบนิยามของ Class ที่ระบุ
NoSuchFieldError(java.lang)	ไม่พบ Field ที่ระบุ
NoSuchMethodError(java.lang)	ไม่พบ Method ที่ระบุ



ประเภทของ Exception

Exception	คำอธิบาย
AWTException	มีปัญหาก่อเกิดขึ้นใน Class ของแพ็คเกจ "java.awt"
ArithmeticException(java.lang)	เกิดความผิดพลาดในการคำนวณ เช่น การหารด้วยค่า 0
ArrayIndexOutOfBoundsException(java.lang)	ระบุตัวเลขเกินขอบเขตของ Array ที่กำหนดไว้
ArrayStoreException(java.lang)	ใส่ค่าที่ไม่ตรงกับชนิดของข้อมูลที่นิยามไว้ใน Array
BindException(java.net)	ไม่สามารถเชื่อมต่อ (Bound) Socket เข้ากับ Local Address และ Port ได้
ClassCastException(java.lang)	แปลงชนิดข้อมูล (Cast) ไม่ตรงกัน
ClassNotFoundException(java.lang)	หา Class หรือ Interface ที่ระบุไม่เจอ
EOFException(java.io)	พบรหัส End of file ก่อนจบการทำงานของโปรแกรมตามปกติ
FileNotFoundException(java.io)	ไม่พบชื่อ File ที่ระบุ
IOException(java.io)	I/O Operation ไม่สามารถทำงานได้เสร็จสมบูรณ์
RuntimeException(java.lang)	Superclass ของ ทุก ๆ Unchecked Runtime Exceptions

ประเภทของ Exception

RuntimeException เป็นข้อผิดพลาดที่หลีกเลี่ยงได้หากเขียนโปรแกรมถูกต้อง เช่น

- **ArrayIndexOutOfBoundsException** เป็นความผิดพลาดที่เกิดจากการอ้างถึงสมาชิกภายในอาร์เรย์ไม่ถูกต้อง เช่น การอ้างถึงสมาชิกในอาร์เรย์ในตำแหน่งที่มากกว่าสมาชิกของอาร์เรย์ที่กำหนด
- **ArithmeticException** เป็นความผิดพลาดที่เกิดจากการดำเนินการทางคณิตศาสตร์ เช่น การหารจำนวนเต็มด้วย 0
- **NullPointerException** เป็นข้อผิดพลาดที่เกิดการอ้างถึงค่าที่เป็น Null เช่น การเรียกใช้ออบเจกต์ที่ยังไม่ได้ถูกสร้าง คือ ค่าออบเจกต์เป็น Null
- **IOException** เป็นข้อผิดพลาดที่ภาษา Java กำหนดให้ต้องมีการจัดการ หากมีการเรียกใช้ เมธอดที่อาจเกิดข้อผิดพลาดประเภทนี้ได้ เช่น
 - **EOFException** เป็นความผิดพลาดที่เกิดจากการระบุจุดสิ้นสุดของไฟล์ไม่ถูกต้อง
 - **FileNotFoundException** เป็นความผิดพลาดที่เกิดจากการไม่พบไฟล์ที่ต้องการ



การจัดการข้อผิดพลาดด้วย Exception

การใช้งานคำสั่ง **try – catch** เป็นการจัดการ Exception ที่มีหลักการทำงานคล้ายกับคำสั่ง if statement คือ ถ้าเกิดข้อผิดพลาดในคำสั่งที่ตามหลังคำสั่ง **try** โปรแกรมจะตรวจสอบคำสั่ง **catch** ว่าเป็นข้อผิดพลาดประเภทใด และทำงานที่ชุดคำสั่งหลังคำสั่ง **catch** ที่ตรงกับข้อผิดพลาด มีรูปแบบการใช้งาน ดังนี้

```
try {  
    [statements]  
}  
catch (TheException e) {  
    [statements_n]  
}  
[finalStatements]
```

โดยที่

statements	เป็นชุดคำสั่งที่ต้องการดักจับข้อผิดพลาด
TheException e	เป็นประเภทของ Exception ที่ต้องการดักจับ
statements_n	เป็นชุดคำสั่งที่กำหนดให้ทำงานเมื่อข้อผิดพลาดตรงตาม Exception ซึ่งดักจับข้อผิดพลาดได้มากกว่า 1 ประเภท
finalStatements	เป็นชุดคำสั่งที่ทำงานปกติไม่ว่าจะมี exception เกิดขึ้นหรือไม่



โปรแกรมจัดการข้อผิดพลาดจากการหารด้วยศูนย์ และการป้อนข้อมูลผิดประเภท

```
import java.util.Scanner;
public class InputMisType {
    public static void main (String[] args){
        try {
            System.out.print ("Enter X: ");
            Scanner scan = new Scanner(System.in);
            int X = scan.nextInt();
            System.out.print ("Enter Y: ");
            int Y = scan.nextInt();
            System.out.println (X / Y);
        }
        catch (ArithmeticException e1) {
            System.out.println(e1+"\nหมายถึง \"ข้อผิดพลาด\" + \"ที่เกิดจากการหารด้วยศูนย์\");
        }
        catch (Exception e2) {
            System.out.println (e2+"\nหมายถึง \"ข้อผิดพลาด\" + \"ที่เกิดจากการป้อนข้อมูลเข้าผิดประเภท\");
        }
    }
}
```


โปรแกรมจัดการข้อผิดพลาดจากการหารด้วยศูนย์และการป้อนข้อมูลผิดประเภท

```
> run InputMisType
```

```
Enter X: 33
```

```
Enter Y: 11
```

```
3
```

```
> run InputMisType
```

```
Enter X: 5
```

```
Enter Y: 0
```

```
java.lang.ArithmeticException: / by zero  
หมายถึง "ข้อผิดพลาดที่เกิดจากการหารด้วยศูนย์"
```

```
> run InputMisType
```

```
Enter X: 4
```

```
Enter Y: s
```

```
java.util.InputMismatchException  
หมายถึง "ข้อผิดพลาดที่เกิดจากการป้อนข้อมูลเข้าผิดประเภท"
```



โปรแกรมจัดการข้อผิดพลาดจากการเข้าถึงอาร์เรย์ในตำแหน่งที่ไม่มีอยู่จริง

```
public class OutofBoundTest {  
    public static void main (String[] args){  
        int s[] = new int[2];  
        try {  
            for(int i = 0; i < 3; i++){  
                s[i] = i;  
                System.out.println(s[i]);  
            }  
        }  
        catch(ArrayIndexOutOfBoundsException e) {  
            System.out.println ("Program has some problem with " + e);  
        }  
        System.out.println("End");  
    }  
}
```

> run OutofBoundTest

0

1

Program has some problem with java.lang.ArrayIndexOutOfBoundsException: 2

End

โปรแกรมจัดการข้อผิดพลาดจากการเปิดไฟล์ไม่มีอยู่จริง

```
import java.io.*;
public class IOExceptType {
    public static void main (String[] args) {
        FileInputStream f_in = null;
        File fname = new File("d:\\ByteData.txt");
        try {
            f_in = new FileInputStream(fname);
            System.out.println("Read Data from File : " + fname);
            int c;
            while ((c = f_in.read()) != -1){
                System.out.println(c);
            }
        }
        catch (IOException e){
            System.out.println(e+"\nหมายถึง \"ข้อผิดพลาด\" + \"ที่เกิดจากไม่พบไฟล์ข้อมูลที่ต้องการ\"");
        }
    }
}
```



โปรแกรมจัดการข้อผิดพลาดจากการเปิดไฟล์ไม่มีอยู่จริง

> **run IOExceptType**

Read Data from File : d:\ByteData.txt

55

48

> **run IOExceptType**

java.io.FileNotFoundException: d:\ByteData.txt (The system cannot find the file specified)

หมายถึง "ข้อผิดพลาดที่เกิดจากไม่พบไฟล์ข้อมูลที่ต้องการ"



การจัดการข้อผิดพลาดด้วย Exception

การใช้งาน **finally** ในคำสั่ง **try – catch finally** เป็นบล็อกการทำงานที่นำมาต่อท้ายไว้กับ try-catch ซึ่งคำสั่งที่อยู่ภายในบล็อก finally จะถูกเรียกใช้งานเสมอ ไม่ว่าจะเกิด Exception หรือไม่ มีรูปแบบการใช้งานดังนี้

```
try {  
    [statements]  
}  
catch (TheException e) {  
    [statements_n]  
}  
finally {  
    [finalStatements]  
}
```

โดยที่

statements	เป็นชุดคำสั่งที่ต้องการดักจับข้อผิดพลาด
TheException e	เป็นประเภทของ Exception ที่ต้องการดักจับ
statements_n	เป็นชุดคำสั่งที่กำหนดให้ทำงานเมื่อข้อผิดพลาดตรงตาม Exception ซึ่งดักจับข้อผิดพลาดได้มากกว่า 1 ประเภท
finalStatements	เป็นชุดคำสั่งที่ทำงานปกติไม่ว่าจะมี exception เกิดขึ้นหรือไม่



โปรแกรมจัดการข้อผิดพลาดเมื่อทำการใส่ finally

```
import java.io.*;
public class FinallyTest {
    public static int num;
    public static void main (String[] args){
        InputStreamReader ir = new InputStreamReader(System.in);
        BufferedReader in = new BufferedReader(ir);
        try {
            System.out.print ("Please input number : ");
            num = Integer.parseInt(in.readLine());
        }
        catch (IOException e1) {
            System.out.print ("Keyboard input Error"); }
        catch (NumberFormatException e2) {
            System.out.println ("Input error, please input number only"); }
        finally {
            System.out.println ("Important : You must input number only"); }
        System.out.println ("Your number is "+ num);
    }
}
```

โปรแกรมจัดการข้อผิดพลาดเมื่อทำการใส่ finally

> run FinallyTest

Please input number : **222**

Important : You must input number only

Your number is 222

> run FinallyTest

Please input number : **cpe**

Input error, please input number only

Important : You must input number only

Your number is 0



การจัดการข้อผิดพลาดด้วย Exception

การใช้งานคำสั่ง **throws** เป็นคำสั่งส่งข้อผิดพลาดออกไป มีรูปแบบการใช้งาน ดังนี้

```
[modifier] return_type MethodName([parameter]) throws ExceptionType1  
[,ExceptionType2] {  
    [statements]  
    return varValue;  
}
```

โดยที่

modifier	เป็นคีย์เวิร์ดที่กำหนดคุณสมบัติการเข้าถึงเมธอด
return_type	เป็นชนิดของข้อมูลที่เมธอดจะส่งค่ากลับ
MethodName	เป็นชื่อเมธอด
parameter	เป็นชุดตัวแปรที่ใช้ในการรับข้อมูล
ExceptionType1, ExceptionType2	เป็นประเภทของข้อผิดพลาดที่ต้องการตรวจจับ
statements	เป็นชุดคำสั่งกำหนดการทำงานของเมธอด
varValue	เป็นค่าที่ต้องการส่งค่ากลับ

การใช้คำสั่ง **throws** จะส่งต่อข้อผิดพลาดไปเรื่อยๆ โดย เมธอด **main()** จะเป็นเมธอดสุดท้ายที่ใช้คำสั่ง **throws**



โปรแกรมแสดงการโยน Exception ออกด้วยคำสั่ง throw

```
public class ThrowsExceptionTest{
    public static void main (String[] args){
        ThrowsExceptionTest t = new ThrowsExceptionTest();
        t.method1(6,3);
        t.method1(6,0);
        t.method1(6,2.5); }
    public void method1(double num1, double num2){
        System.out.println(method2(num1,num2));
    }
    public String method2(double a, double b) throws ArithmeticException{
        if(b==0){
            System.out.println(a+ "/" +b+ " ===> ");
            throw new ArithmeticException();
        }
        else{
            return a+ "/" +b+ " = " +(a/b);
        }
    }
}
```

โปรแกรมแสดงการโยน Exception ออกด้วยคำสั่ง throw

```
> run ThrowsExceptionTest
```

```
6.0/3.0 = 2.0
```

```
6.0/0.0 ==>
```

```
java.lang.ArithmeticException
```

```
    at ThrowsExceptionTest.method2(ThrowsExceptionTest.java:13)
```

```
    at ThrowsExceptionTest.method1(ThrowsExceptionTest.java:8)
```

```
    at ThrowsExceptionTest.main(ThrowsExceptionTest.java:5)
```

```
    at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
```

```
    at sun.reflect.NativeMethodAccessorImpl.invoke(Unknown Source)
```

```
    at sun.reflect.DelegatingMethodAccessorImpl.invoke(Unknown Source)
```

```
    at java.lang.reflect.Method.invoke(Unknown Source)
```

```
    at
```

```
edu.rice.cs.drjava.model.compiler.JavacCompiler.runCommand(JavacCompiler.java:272)
```



โปรแกรมแสดงการโยน Exception ออกด้วยคำสั่ง throw 1

```
public class ThrowsExceptionTest1{
    public static void main (String[] args){
        ThrowsExceptionTest t1 = new ThrowsExceptionTest1();
        t1.method1(6,3);
        t1.method1(6,0);
        t1.method1(6,2.5); }
    public void method1(double num1, double num2){
        try{
            System.out.println(method2(num1,num2)); }
        catch(ArithmeticException exception){
            System.out.println ("Divide by zero not allowed "); } }
    public String method2(double a, double b) throws ArithmeticException{
        if(b==0){
            System.out.println(a+ "/" +b+ " ==> ");
            throw new ArithmeticException(); }
        else{
            return a+ "/" +b+ " = " +(a/b); }
    }
}
```

> run ThrowsExceptionTest1

6.0/3.0 = 2.0

6.0/0.0 ==>

Divide by zero not allowed

6.0/2.5 = 2.4

โปรแกรมแสดงการโยน Exception ออกด้วยคำสั่ง throw 2

```
public class ThrowsExceptionTest2{
    public static void main (String[] args){
        ThrowsExceptionTest2 t = new ThrowsExceptionTest2();
        try{
            t.method1(6,3);
            t.method1(6,0);
            t.method1(6,2.5);    }
        catch(ArithmeticException exception){
            System.out.println ("Divide by zero not allowed ");    }
    }
    public void method1(double num1, double num2) throws ArithmeticException{
        System.out.println(method2(num1,num2));    }
    public String method2(double a, double b) throws ArithmeticException{
        if(b==0){
            System.out.println(a+ "/" +b+" ==> ");
            throw new ArithmeticException();    }
        else{
            return a+ "/" +b+" = " +(a/b);    }
    }
}
```

> run ThrowsExceptionTest2

6.0/3.0 = 2.0

6.0/0.0 ==>

Divide by zero not allowed

การสร้างและใช้งาน Exception ที่สร้างขึ้นเอง

exception มีอยู่หลายประเภทด้วยกัน นอกจากนี้แล้ว เรายังสามารถกำหนดประเภทของ exception ขึ้นมาใช้งานได้ โดยนิยามคลาสใดๆ ให้สืบทอดมาจากคลาส exception ซึ่งมี constructor 2 แบบ คือ

- `public Exception()`
- `public Exception(String s)`

ในการดักจับ exception ที่เราสร้างขึ้นมาเองสามารถทำได้ด้วยการใช้ try-catch ปกติ เมื่อเกิดข้อผิดพลาดขึ้นในคำสั่งใด เมธอดที่ต้องการส่งมอบเจ็กต์ประเภท exception จะต้องเรียกใช้ คำสั่งที่ชื่อ throws เพื่อจะสร้างมอบเจ็กต์ของคลาสประเภท exception ขึ้นมา และเมธอดนั้นจะต้องมีคำสั่ง throws เพื่อให้เมธอดที่เรียกใช้เมธอดนี้จัดการข้อผิดพลาดที่เกิดขึ้น



โปรแกรมจัดการข้อผิดพลาด โดยใช้ Exception ที่สร้างขึ้นเอง

```
import java.util.Scanner;
public class OutOfRangeException extends Exception {
    public void checkvalue(int value, int MIN, int MAX)
        throws OutOfRangeException {
        if (value < MIN || value > MAX){
            throw new OutOfRangeException();
        }
    }
}

public class CreatingExceptions {
    public static void main (String[] args) {
        OutOfRangeException x = new OutOfRangeException();
        final int MIN = 0, MAX = 30; int value=0;
        try {
            Scanner scan = new Scanner (System.in);
            System.out.print ("Enter Score: ");
            value = scan.nextInt();
            x.checkvalue(value,MIN,MAX);
        }
    }
}
```



โปรแกรมจัดการข้อผิดพลาด โดยใช้ Exception ที่สร้างขึ้นเอง

```
catch (OutOfRangeException e){
    System.out.println(e + "\nหมายถึง \"ข้อผิดพลาดที่เกิดจากค่าข้อมูลคะแนนเกิน
ขอบเขต\"");
    System.exit(0);
}
catch (Exception e) {
    System.out.println(e + "\nหมายถึง \"ข้อผิดพลาดที่เกิดจากการป้อนข้อมูลเข้าผิด
ประเภท\"");
    System.exit(0);
}
finally {
    System.out.println("คะแนนคือ " + value*100.0f/MAX + "%");
}
}
```



โปรแกรมจัดการข้อผิดพลาด โดยใช้ Exception ที่สร้างขึ้นเอง

> run CreatingExceptions

Enter Score: 23

คะแนนคือ 76.666664%

> run CreatingExceptions

Enter Score: 40

OutOfRangeException

หมายถึง "ข้อผิดพลาดที่เกิดจากค่าข้อมูลคะแนนเกินขอบเขต"

คะแนนคือ 133.33333%

> run CreatingExceptions

Enter Score: s

java.util.InputMismatchException

หมายถึง "ข้อผิดพลาดที่เกิดจากการป้อนข้อมูลเข้าผิดประเภท"

คะแนนคือ 0.0%

